Who Wrote It?

Identifying LLM source configurations from response prefixes

John Kushwarra · Individual participant and project lead
CMPSC 448 · Machine Learning and Algorithmic AI

1. Problem and data

This project asks whether a classifier can identify which LLM generated a response. The target has three labels: Gemma, Mistral and Qwen. Following the assignment, a CNN and an LSTM were tested using the prompt alone, the response alone, and both together. Two follow-up questions examine task transfer and distinguishing text patterns.

Source and selection

The data came from Ai2's public Dolci DPO Model Response Pool [1], downloaded through the Hugging Face Dataset Viewer on October 8, 2026. The first 1,500 rows from each configuration below were matched by source ID and identical prompt text. The retained examples all belong to the DaringAnteater-prefs_olmo2_7b subset. This is a convenience sample, not a random sample of the full dataset.

Source configuration	Target	Original responses
gemma3-12b	Gemma	827
mistral-24b	Mistral	827
qwen3-no_reasoning-4b	Qwen	827

Table 1. One specific source configuration represents each family.

Whole prompt groups were excluded for record-extraction/JSON-schema prompts (476), prompt length outside 30-4,000 characters (152), response length outside 50-20,000 characters (34), or an email pattern (11). Removing these 673 groups left 827 prompts and 2,481 responses, with one response per family for each prompt. Each row includes LLM_name, LLM_Input and LLM_output.

Length limits avoid very short examples with little text to compare and unusually large records. Removing selected extraction/schema tasks reduces rigid output formats. The email check is a basic precaution against retaining contact information.

Keeping related examples together

The three responses to a prompt were assigned together to training, validation or test using seed 448. This keeps responses to the same matched prompt out of different splits. Each split is exactly balanced across the three labels.

Partition	Prompt groups	Gemma	Mistral	Qwen	Responses
Training	578	578	578	578	1,734
Validation	124	124	124	124	372
Test	125	125	125	125	375

Table 2. Fixed prompt-group split; no group crosses partitions.

Exact duplicate prompts and responses were checked across splits; paraphrases may still remain. The source card lists ODC-BY and links model-output terms [1]. Only public data was used. Filtering reduced unsuitable records but was not an exhaustive content review.

2. Models and training

The CNN and LSTM were implemented in PyTorch. An LSTM is a type of recurrent neural network (RNN). The dataset records which source configuration generated each response. That label is what the classifiers predict; the generating LLMs are not trained in this project. The same case-insensitive mask removes literal Gemma, Mistral and Qwen names. Model IDs and other label metadata are never included as features.

Text is split into case-sensitive word and punctuation tokens. A vocabulary of 12,000 entries is built from training text only, with special tokens for padding, unknown words and a separator. Each input field keeps its first 128 tokens. The combined condition keeps 128 prompt tokens, a separator, and the same 128 response tokens, so adding a prompt cannot displace response text.

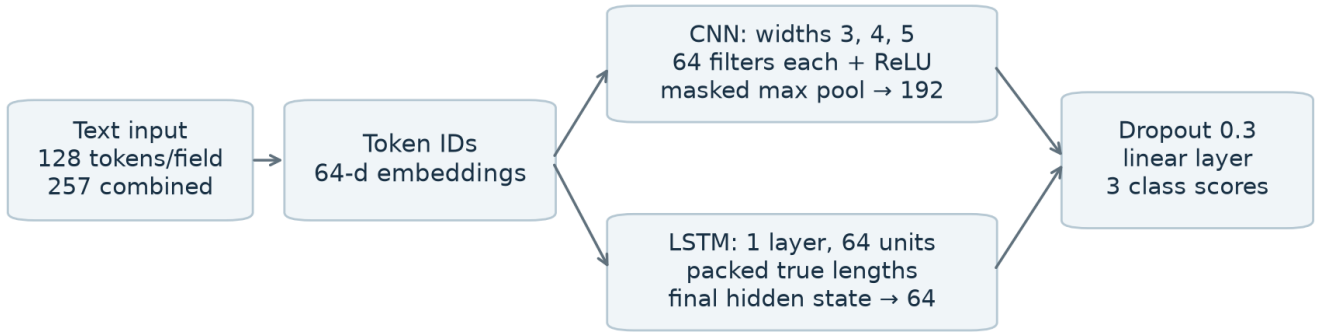


Figure 1. Both classifiers map token IDs to 64-dimensional embeddings, then to three class scores. The CNN combines local patterns; the LSTM processes a sequence.

The CNN uses 64 filters at each width of 3, 4 and 5 tokens, followed by ReLU, max pooling, dropout and a linear output layer. Pooling keeps each filter's strongest response to a pattern, producing 192 features. The single-layer LSTM uses 64 hidden units and its final hidden state, followed by dropout and a linear layer. Both models ignore the padding added to shorter sequences.

Setting	Value
Embedding / model size	Random 64-d embeddings; 64 filters per CNN width / 64 LSTM units
Objective / optimizer	Cross-entropy; Adam, learning rate 0.001
Batch / regularization	Batch size 64; dropout 0.3; gradient norm clipped at 1.0
Selection / stopping	Best validation loss; stop after 3 consecutive epochs without sufficient improvement; maximum 12 epochs
Compute / seed	CPU, 2 threads, deterministic PyTorch operations; seed 448
Extra investigations	RQ3/RQ4 repeat seeds 448, 449 and 450

Table 3. Training settings shared by the initial CNN/LSTM comparisons.

The saved model is the checkpoint with the lowest validation loss, rather than automatically the final epoch. The original test set is evaluated after selection. Accuracy is the fraction predicted correctly; macro F1 gives each of the three classes equal weight. Predicting one class for every balanced test example gives 33.3% accuracy and 0.167 macro F1.

3. Required results: RQ1 and RQ2

All six conditions used the same 375 test responses (125 prompt groups) and training seed 448.

Model	Input	Accuracy (%)	Macro F1	Epoch
CNN	Prompt only	33.3	0.285	8
CNN	Response only	91.7	0.917	11
CNN	Prompt + response	89.3	0.894	11
LSTM	Prompt only	33.3	0.319	3
LSTM	Response only	73.1	0.727	10
LSTM	Prompt + response	74.7	0.746	10
Majority	Training-label baseline	33.3	0.167	-

Table 4. Test accuracy, macro F1 and validation-selected epoch for all six required conditions.

RQ1: can responses identify their source?

Response-only accuracy was 91.7% for the CNN and 73.1% for the LSTM, both above the 33.3% baseline. The CNN performed better under these training settings.

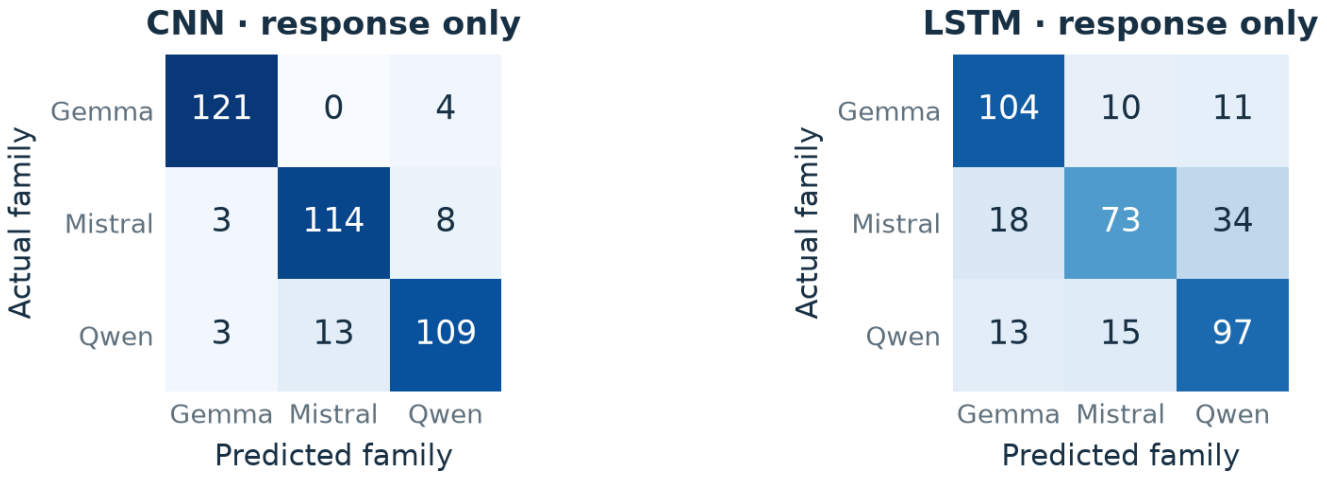


Figure 2. Response-only confusion matrices. Rows are actual labels and columns are predicted labels; each row contains 125 responses.

The largest directional errors were Qwen predicted as Mistral for CNN (13/125), and Mistral predicted as Qwen for LSTM (34/125). Results therefore vary by class.

RQ2: does adding the prompt help?

Each prompt appears once under every label. A deterministic prompt-only classifier predicts the same label for all three copies, so it gets exactly one right: 33.3%. A higher score here would suggest broken matching, leakage or an evaluation error; prompts can be informative in other datasets.

Adding prompts changed CNN accuracy from 91.7% to 89.3% and LSTM from 73.1% to 74.7%. These small changes do not establish a reliable improvement from adding prompts.

4. RQ3: transfer between tasks

Question and setup

Would a model trained on one kind of request recognize sources on another? Prompt-based rules separated question-answering requests from drafting/artifact requests and excluded ambiguous cases. The assistant checked 20 sampled prompt openings per category before training. These are broad task forms with overlapping topics, rather than independently labeled domains.

Each task supplied 110 training and 29 validation prompt groups. The tests contained 25 question groups (75 responses) and 41 drafting groups (123 responses). Both architectures were trained on each task with seeds 448, 449 and 450. Vocabulary building and checkpoint selection used the source task only. Every trained model was then tested on both tasks.

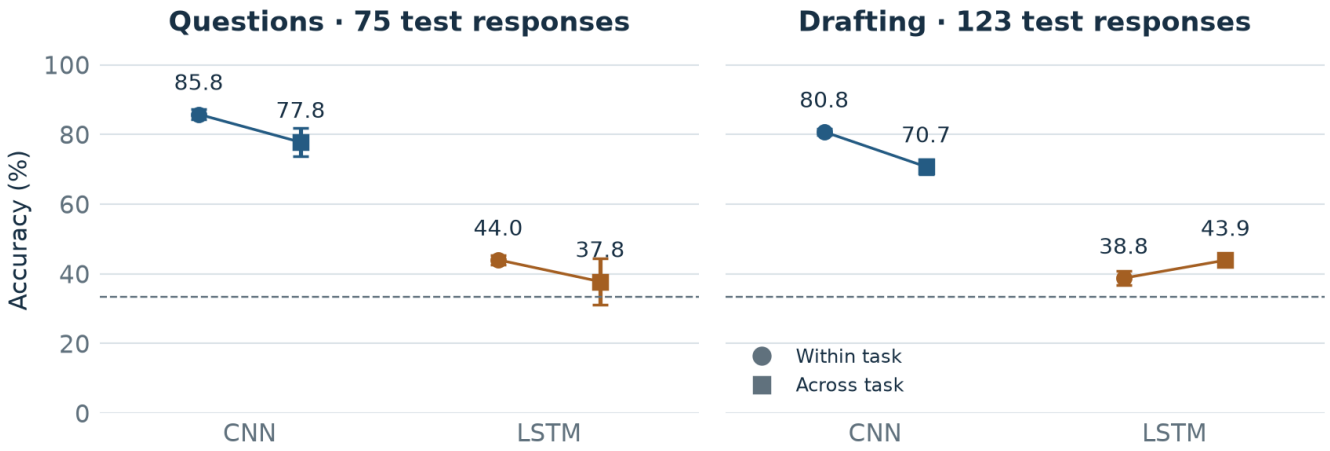


Figure 3. Compare within-task and across-task training on the same target test. Points show mean accuracy over three seeds; bars show one seed standard deviation, not a confidence interval.

What changed?

On the question test, the CNN averaged 85.8% when trained on questions and 77.8% when trained on drafting: an 8.0 percentage-point loss. On the drafting test, within-task training gave 80.8%, compared with 70.7% after training on questions: a 10.0-point loss. Differences use unrounded results.

The LSTM was weak even within task: 44.0% on questions and 38.8% on drafting. Across-task scores were 37.8% and 43.9%, respectively. Similar low scores provide little evidence of robust transfer. Gemma had the highest or tied-highest CNN recall in each source/target combination, but that pattern is limited to this sample.

Interpretation

The CNN retained useful source information across tasks, but its accuracy fell when the task changed. This suggests that some learned patterns depend on the kind of request. The small tests and rule-based task labels limit how far this result can be generalized. Per-class results and individual-seed scores remain in results/rq34_v1/.

5. RQ4: signals in the responses

Observed differences

The training responses differed in length and formatting. Median full-response length was about 5,637 characters for Gemma, 2,958 for Mistral and 2,293 for Qwen. Punctuation made up an average of 26.7%, 25.0% and 20.3% of visible prefix tokens, respectively. The word 'okay' appeared in 257/578 Gemma training prefixes and none of the other families' prefixes. These are sample associations, not universal traits of the families.

Replacing words or punctuation

Both models were retrained after replacing either all words and numbers or all punctuation and symbols with one placeholder token. Token positions, sequence lengths, vocabulary size and training settings stayed fixed. This tests how much the models can learn from the patterns that remain.

Training input	CNN accuracy (%)	LSTM accuracy (%)
Original response prefix	90.2 +/- 1.3	76.4 +/- 3.3
Words/numbers replaced	89.4 +/- 0.8	78.1 +/- 1.7
Punctuation/symbols replaced	78.8 +/- 2.0	58.0 +/- 6.9

Table 5. Test accuracy (%) after retraining, shown as mean +/- one seed standard deviation across seeds 448, 449 and 450.

The CNN still reached 89.4% when words and numbers were replaced, close to its three-run average of 90.2% on unchanged text. Replacing punctuation and symbols reduced it to 78.8%. The LSTM also lost more accuracy when punctuation and symbols were replaced. Punctuation and its position in the text therefore helped distinguish the sources in this dataset.

What this does and does not show

An additional check, done after the main experiments, compared 116 test groups (348 responses) where all three responses had the same visible length of 128 tokens. The CNN trained with words and numbers replaced still reached 90.5%, compared with 91.3% for the CNN trained on unchanged text, using the same examples.

These replacements leave some stylistic clues: punctuation positions remain visible even when the actual symbols are replaced. Full-response length and original line breaks are not explicit model inputs. The results show that text structure helps distinguish these sources, but they do not show that meaning never matters or that one particular word causes a prediction.

6. Additional classifiers and limitations

Comparing individual models and combinations

The observed text patterns motivated a comparison using TF-IDF features, which represent text using weighted counts of words or short character sequences. Logistic regression and linear support vector machines (SVMs) from scikit-learn were tried, along with combinations of the neural models. All received the same 128-token response text with model names replaced by a shared placeholder. The table focuses on the original CNN and the individual model and ensemble chosen using validation data. Full comparisons remain with the code.

The 124 validation groups were split in half: one half adjusted the models' confidence scores, and the other selected settings and weights. Prediction scores from three CNN training runs were averaged, as were scores from three LSTM runs. The final combination gave 25% weight to each average and 50% to the SVM; the highest combined score determined the predicted label. These choices were fixed before testing on 59 new matched prompt groups (177 responses) from the same source. This confirmation batch was not used to train or choose the models.

Classifier	Original accuracy (%)	Confirmation accuracy (%)
Original CNN (seed 448)	91.7	85.9
Selected word + character SVM	93.9	91.5
Selected CNN/LSTM/SVM ensemble	93.9	89.8

Table 6. Selected comparisons. Original test: 375 responses; confirmation: 177. The original test had already informed exploratory follow-ups.

The ensemble gained only one correct response over the SVM during selection, but scored 89.8% on confirmation versus 91.5% for the SVM. It did not retain that small advantage. These results do not establish a reliable difference between the two.

One example of a shared mistake

A Mistral response contained only 15 tokens: '- Katana (Japan) - Claymore (Scotland) - Gladius (Roman)'. The prompt required three dashed list items. CNN, LSTM and SVM all predicted Qwen. This example comes from the errors examined after testing [1]. A short, constrained answer offers limited stylistic evidence, although one case cannot establish a general effect of response length.

Main limitations

The results concern three specific configurations from one selected portion of a public dataset. Most responses were shortened: 364/375 original test responses exceeded 128 tokens. The initial comparison used one training seed, and later experiments followed inspection of the original test. The neural models had already used the full validation set to choose checkpoints before it was split for adjusting confidence scores and selecting the ensemble. The confirmation sample is small and from the same source, and examples with very similar meanings may remain. These results do not establish performance on unseen families or arbitrary human/AI writing.

7. Lessons and conclusion

Practical challenges

Several details affected whether the comparison was fair. Keeping prompt groups together reduced leakage, while fitting the vocabulary on training data kept test information out of preprocessing. Separate token allowances ensured that adding a prompt did not shorten the visible response. Explicit padding handling prevented artificial padding tokens from affecting CNN pooling or LSTM states.

Model selection also mattered. Checkpoints were chosen using validation loss, and ensemble settings were fixed before confirmation. The selected ensemble's small validation gain did not persist. This makes the distinction between choosing a model and evaluating it especially clear.

Personal reflection

Combining classifiers was the part of the project that stood out to me. I developed a clearer understanding of how voting and weighted averages combine predictions, and why adding classifiers does not guarantee better accuracy. The selected ensemble gained only one correct response over the selected SVM on validation, then scored 89.8% compared with the SVM's 91.5% on confirmation. This provided a concrete example of why a combination needs to be evaluated rather than assumed to be better.

Assistance and responsibility

This individual project used assistance from OpenAI Codex for multiple parts of this project

Conclusion

The response-only CNN performed better than the initial LSTM on this sample. Adding prompts gave no consistent improvement, changing tasks reduced CNN accuracy, and punctuation/position patterns retained useful source information even when word identities were removed. A simple TF-IDF SVM was also competitive, while the selected ensemble failed to preserve its small validation advantage.

The project shows that these sampled source configurations can be distinguished from short response prefixes. A useful next step would be to test new model versions and a separate data source, with all choices fixed beforehand. That would provide a stronger check of whether the learned patterns generalize.

References and supporting material

References

[1] Ai2. Dolci DPO Model Response Pool. Dataset card and model-output terms. Accessed October 8, 2026. The card requests attribution to Team Olmo et al., Olmo 3 (2025), arXiv:2512.13961. Source link.

Code and result summaries

The accompanying repository contains the data collection and training code, this report, and result summary tables and figures. The README explains how to run new experiments, and the dataset and results notes describe the source data and comparisons.

Reproducing the results

The project used Windows, Python 3.12.10 and CPU PyTorch, with exact dependencies in requirements-lock.txt. A previous check reproduced the saved predictions in a separate environment; it did not retrain all models. Datasets, saved models and detailed run files are not included in the repository. The data collection scripts use the public source, but new downloads may differ. The repository alone therefore cannot exactly replay the reported results.